code-conventions | Status: mixed | App v1.0.20 | Reviewed 2026-10-24

Code Conventions

These conventions were **observed** across the 326 Kotlin files in `gallery/` **[current]**. There is no formatter or linter config in the repo, so they are enforced by review only. Items marked **[proposed]** are recommendations.

1. Files and headers

- Every Kotlin, Gradle, and XML source starts with the Apache-2.0 header (`Copyright <year> Google LLC`). All 326 Kotlin files have it. New files use the current year.
- One primary public type per file, named after the file. Small related types (UI state, params) may share it.
- Package = `com.google.ai.edge.gallery.<area>[.<feature>]`, matching the directory.

2. Formatting

Rule	Evidence
2-space indentation, no tabs	Codebase-wide. One exception: `ui/common/AudioAnimation.kt`.
~100-column lines	Only about 30 code lines go over 100 characters (excluding strings, URLs, imports)
Trailing commas in multi-line parameter and argument lists	Pervasive
Expression bodies and named arguments for multi-parameter calls	Pervasive
`kotlin.code.style=official`	`gradle.properties`

This matches **ktfmt (Google style)** output **[inferred]**. **[proposed]** Add ktfmt (Google style) through Spotless or a pre-commit hook so the style is enforced automatically.

3. Naming

Kind	Convention	Examples
Interface + default impl	`Foo` + `DefaultFoo`	`DataStoreRepository` / `DefaultDataStoreRepository`, `LlmSessionManager` / `DefaultLlmSessionManager`, `DownloadRepository` / `DefaultDownloadRepository`

Kind	Convention	Examples
In-memory / runtime impl	`RuntimeFoo`	`RuntimeToolsProvider`, `RuntimeToolDispatcher`
Disabled impl	`NoOpFoo`	`NoOpSkillsProvider`
Feature task	`<Feature>Task` + `<Feature>TaskModule`	`SmartAlbumTask`, `SmartAlbumTaskModule`
Screen / VM	`<Feature>Screen`, `<Feature>ViewModel`	`AgentChatScreen`, `McpManagerViewModel`
Model helper	`<Runtime>ModelHelper` (singleton `object`)	`LlmChatModelHelper`, `AICoreModelHelper`
DataStore serializer	`<Proto>Serializer` (`object`)	`SettingsSerializer`, `SkillsSerializer`
Constants	`UPPER_SNAKE_CASE` top-level `const val`. WorkManager keys start with `KEY_`.	`DEFAULT_MAX_TOKEN`, `KEY_MODEL_URL`
Built-in task ids	`snake_case` strings in `BuiltInTaskId`	`llm_agent_chat`
Analytics events	`snake_case` ids in `GalleryEvent`	`mcp_execution`
Log tags	`private const val TAG = "AG<ClassName>"` (85 files)	`AGModelManagerViewModel`

4. Dependency injection (Hilt)

- Modules: `@Module @InstallIn(SingletonComponent::class) internal object XModule`. Providers are `@Provides` (+ `@Singleton`).
- New features register with `@Provides @IntoSet fun provideTask(...): CustomTask`. Don't edit a central list.
- Strategies keyed by enum use map multibinding (`@IntoMap @RuntimeTypeKey(RuntimeType.X)`).
- Qualifiers: dispatchers (`@IoDispatcher`, `@DefaultDispatcher`, `@MainDispatcher`), `@ApplicationScope`, executors (`@AiChatExecutor`, `@AgentChatExecutor`).
- Inject `@ApplicationContext context: Context`. Never inject an Activity context into singletons.

5. State and UI

- ViewModels: `@HiltViewModel` + `@Inject constructor`. State is an immutable `data class XUiState`, held as `protected/private val _uiState = MutableStateFlow(...)` and exposed as `val uiState = _uiState.asStateFlow()`. Mutate with `_uiState.update { it.copy(...) }`.
- Composables take a ViewModel parameter defaulting to `hiltViewModel()` and collect state with `collectAsState()`. `collectAsStateWithLifecycle()` is used in 10 files. **[proposed]** Prefer `collectAsStateWithLifecycle()` in new code.
- User-facing text uses `stringResource(R.string.…)`. Each string has a `description` with `[CHAR_LIMIT=n]` for translators. Product names use `translatable="false"`. About 17 files still have hard-coded `Text("…")`. **[proposed]** Move these to resources when you touch them.
- Use `sealed interface` for state machines and event hierarchies (`Model.InitializationStatus`, `AgentEvent`).
- `@Preview` exists in about 20 files. Add a preview for reusable components.

6. Async and errors

- Coroutines everywhere. Prefer **injected** dispatchers (`@IoDispatcher`) over `Dispatchers.IO` (35 files still use `Dispatchers.IO` directly; only 6 inject it). **[proposed]** Inject dispatchers in new code so it can be tested.
- Callback convention for model lifecycle: `onDone(error: String)`, where `""` means success and anything else is the error message. Don't throw across this boundary.
- Tools return `Map<String, String>` with `"status"` = `"succeeded"` / `"failed"` plus `"result"` or `"error"`. The model reads these, so messages should guide its next step.
- `runBlocking` appears in DataStore access and tool bodies, where LiteRT-LM calls tools synchronously. Don't add new `runBlocking` on the main thread.

7. Documentation in code

- KDoc on public interfaces and data classes, with `@param` / `@property` (≈950 KDoc blocks). Interfaces such as `CustomTask`, `AgentRuntimeExecutor`, `LlmSessionManager`, and `LlmModelHelper` document their contracts in detail. Keep them current.
- Inline comments explain **why** (hardware quirks, e.g. "must be GPU for Gemma 3n", workarounds with issue links).
- TODOs: `// TODO: …` or `// TODO(owner): …` (10 in the tree).

8. Data and persistence

- Persisted state is Proto DataStore, with the schema in `app/src/main/proto/*.proto`. Add fields with new tag numbers and never reuse removed ones. A `DataStore<T>` is provided once, as a singleton in a Hilt module.
- JSON from the network or the model: Gson for the allowlist, Moshi (`@JsonClass(generateAdapter = true)`) for tool and intent payloads.
- Never put real credentials in `ProjectConfig` or `build.gradle.kts`. The `REPLACE_WITH_…` placeholders are intentional.

9. Git and change hygiene

- Commit subject: a single imperative sentence that ends with a period, e.g. "Avoid calling createConversation twice during model initialization." Many commits are titled "Internal changes" and authored by "Google AI Edge Gallery", which suggests an automated export from an internal repository **[inferred]**.
- External code contributions aren't accepted yet (CONTRIBUTING.md).
- Release versions are git tags equal to `versionName` (e.g. `1.0.20`).
- When you change a file listed in a doc's `sources`, update that doc in the same change (see Docs/README).